```
title: "Report Lab 1: Bayesian Networks - lingo978, mancl774, vikno856"
output:
  pdf_document:
    latex_engine: xelatex
```

```
knitr::opts_chunk$set(echo = FALSE, message=FALSE)
knitr::read_chunk("R/group.R")
```

Question 1

Show that multiple runs of the hill-climbing algorithm can return non-equivalent Bayesian network (BN) structures. Explain why this happens.

Answer. All three variants of hill climbing (different score functions, different imaginary sample sizes, and different random starting structures) return structures that are not equivalent. This is shown by the plots and by converting the DAGs to equivalence classes by using the `cpdag()` function and comparing them using `all.equal()`, meaning the two DAGs do not belong to the same equivalence class.

Hill climbing is a greedy local search that repeatedly accepts the single edge change giving the largest improvement in the score. Because the search is greedy, it can get stuck in different local optima, so the returned structure is not guaranteed to be the global optimum.

Question 2

Learn a BN from 80 % of the Asia dataset. Use the BN learned to classify the remaining 20 % of the Asia dataset in two classes: S = yes and S = no. In other words, compute the posterior probability distribution of S for each case and classify it in the most likely class.

Answer. The data from the Asia dataset is split into 80% training data and 20% testing data.

The training data is used to learn a BN structure by running the Hill-Climbing algorithm. The given "truth" structure is also defined.

Resulting in the following networks:

This learned structure and the given "truth" structure are then used to learn the parameters according to the training data using exact inference.

The parameters are then used as parameters to this function, which does the following to each row in the test data:

- Inserts the observed variables (every variable present in the data expect the target variable S) as evidence in the given network.
- Uses exact inference to compute the posterior probability of the target variable S.
- Given the posterior probability, S is classified as the state (yes/no) with the highest probability.

Running this function results in the following confusion matrix for the learned structure:

And the following confusion matrix for the given structure:

Comparing the two matrices, one can conclude that they are equivalent.

Question 3

Now, you are asked to classify S given observations only for the so-called Markov blanket of S

Answer. A modified version of the function in Question 2 is used that takes a set of nodes to use for observation to compute the classification of S for each entry in the test data:

The Markov Blanket for the structures present are used as the second argument for the function, and are extracted using the mb() function:

Running this function with the Markov Blanket for the learned structure results in the following confusion matrix:

Question 4

Repeat the exercise (2) using a naive Bayes classifier

Answer. The naive Bayes network assumes all features are conditionally independent given S. The resulting confusion matrix differs from Question 2, because the naive Bayes assumption removes the dependencies that exist between the features (e.g. $T \rightarrow E$ and $B \rightarrow D$) in the true and the learned network, leading to different posterior estimates of S.

Given the target variable S, the following BN structure is created using a naive Bayes classifier (the target acts as a parent to every predictor, and there are no edges between the predictors):

Question 5

Explain why you obtain the same or different results in the exercises (2-4).

Answer. Using all predictors to predict S and using only the Markov Blanket of S to predict S yields the same result, given the same BN. This makes sense, because whenever the Markov Blanket for S is observed, the other predictors provide no further information about S. Hence, the confusion matrices are equivalent for both cases.

The Naive Bayes structure differs from the one learned by HC. In Naive Bayes, all predictors are conditionally independent given S (since S is a parent fork to all predictors), while the network learned by HC can have dependencies between predictors. This produces a different joint distribution and can therefore produce different posterior probabilities for S. This is what happened in this case, and is why the confusion matrix differs

Appendix

```
cat("```r\n")
cat(readLines("R/group.R"), sep = "\n")
cat("\n```")
```